

Morpho Midnight

Security Review

Cantina Solo review by:
StErMi, Lead Security Researcher

August 10, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Low Risk	4
3.1.1	Liquidators are allowed to leave dust collateral or debt positions	4
3.1.2	Consider reverting the <code>supplyCollateral</code> operation if the collateral's oracle is broken	5
3.2	Informational	5
3.2.1	Bulk Informational Finding	5
3.2.2	Bulk natspec issues	7
3.2.3	Trading fees can be set higher as we approach Market's Time To Maturity	8
3.2.4	Consider exposing a <code>isOfferRatified</code> function in the Midnight contract	8
3.2.5	Liquidation can seize the whole borrower collateral without paying any debt when collateral price is zero	9
3.2.6	Liquidation could end up repaying debt without receiving any collateral back	9
3.2.7	Consider allowing the owner of the nonce (or an authorized user) to increase the <code>EcrecoverAuthorizer</code> 's <code>nonce</code>	9
3.2.8	Consider tracking the <code>signer</code> in the <code>SetIsAuthorized</code> event of the <code>EcrecoverAuthorizer</code> contract	10
3.2.9	The "ROUNDINGS" should document that different <code>units</code> inputs could produce the same results in assets transferred	10
3.2.10	Consider passing the "opposite" side data information to the <code>onBuy</code> and <code>onSell</code> callbacks	13
3.2.11	Considerations on the lenders hostile and adversarial environment	14

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Morpho is a trustless and efficient lending primitive with permissionless market creation.

From May 19th to May 26th the security researchers conducted a review of [midnight](#) on commit hash [a9988d99](#). A total of **13** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	2	0	2
Gas Optimizations	0	0	0
Informational	11	4	7
Total	13	4	9

2.1 Scope

The security review had the following components in scope for [midnight](#) on commit hash [a9988d99](#):

```
src
├── Midnight.sol
├── interfaces
│   ├── ICallbacks.sol
│   ├── IERC20.sol
│   ├── IGate.sol
│   ├── IMidnight.sol
│   ├── IOracle.sol
│   └── IRatifier.sol
├── libraries
│   ├── ConstantsLib.sol
│   ├── EventsLib.sol
│   ├── IdLib.sol
│   ├── SafeTransferLib.sol
│   ├── TickLib.sol
│   └── UtilsLib.sol
├── periphery
│   ├── ConsumableUnitsLib.sol
│   ├── EcrecoverAuthorizer.sol
│   ├── TakeAmountsLib.sol
│   │   └── interfaces
│   │       ├── IEcrecoverAuthorizer.sol
│   │       ├── IERC20Permit.sol
│   │       └── IPermit2.sol
│   └── ratifiers
│       ├── EcrecoverRatifier.sol
│       ├── SetterRatifier.sol
│       └── interfaces
│           ├── IEcrecoverRatifier.sol
│           └── ISetterRatifier.sol
└── libraries
    └── HashLib.sol
```

3 Findings

3.1 Low Risk

3.1.1 Liquidators are allowed to leave dust collateral or debt positions

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Description: Protocols like Aave V3 have dedicated liquidation logic that prevents a liquidator from leaving a borrower with dust-level debt or dust-level collateral unless the liquidation fully clears the debt or fully removes the collateral.

This avoids residual positions that are not profitable to liquidate after accounting for gas costs. In those lending protocols, this is especially important because residual debt can continue accruing interest and may eventually become bad debt if liquidators have no economic incentive to clean it up.

Midnight is less exposed to that exact issue because borrower debt does not passively accrue. Existing dust debt does not automatically grow into bad debt.

However, dust debt/collateral positions are still undesirable:

1. A dust debt position can become bad debt if collateral prices move adversely before it is repaid or liquidated.
2. Suppliers may be left with a residual amount of credit that cannot become withdrawable until the dust debt is repaid or liquidated.
3. Dust collateral can keep a collateral slot active; because liquidation reads all active collateral oracles, a dust collateral with a failing oracle can also hurt liquidation liveness.

The current `market.rcfThreshold` logic **does not** force dust cleanup. It **only allows** the liquidator to bypass the pre-maturity RCF cap and repay more than `maxRepaid` when an RCF-limited liquidation would leave less than `rcfThreshold` of repayable collateral capacity.

Because the check is permissive rather than mandatory, a liquidator can still choose a partial liquidation that leaves **dust debt and/or dust collateral**.

When the loan ends there's the expectation that suppliers would be able to withdraw their funds but if the borrower's position, because of the dust collateral or debt, is not profitable to be liquidated (operation expense vs. collateral value), no liquidator will take that responsibility and lenders won't be able to withdraw the expected face value.

As far as I can see, the whitepaper and NatSpec do not cover the residual dust-debt and dust-collateral scenarios that can still remain after liquidation, nor the consequences those residual positions can create for bad-debt risk, supplier withdrawals, and future liquidation liveness.

Recommendation: Morpho should consider the following:

1. Think about whether there is a way to incentivize liquidators or "force them" during liquidation to prevent dust positions.
2. Think about the concept of "fixed time length" and what are the expectations or obligations that the product has to meet towards them once the maturity ends.
3. Document this scenario and the consequences in the Midnight's contract NatSpec.

Morpho: Added documentation in [PR 991](#).

We won't fix as we think that is ok to not be better than blue on this. RCF deactivation prevents the systemic aspect of RCF that creates smaller and smaller positions, but the fact that people can by themselves create small positions or liquidators leave small positions is not worse than blue (slightly worse because of multi-collateral, but the difference in gas is bounded).

Stermi: Acknowledged.

3.1.2 Consider reverting the `supplyCollateral` operation if the collateral's oracle is broken

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Description: As documented in the "LIVENESS" section of the natspec, when a collateral's oracle reverts, plenty of the Midnight's core features will also break and stop working as expected: `liquidate`, `isHealthy`, `withdrawCollateral` and `take`. The above functions will revert if the borrower has activated at least one of the broken oracles.

Morpho could at least try to avoid the above situation by actively checking if the user is activating a broken collateral when the `supplyCollateral` function is executed.

Let's say that the user has an existing debt and collateral position that works correctly. In this situation the user can withdraw their collateral, can perform `take` operations (to increase his debt or reduce it) and can be liquidated (if maturity has been reached or his not healthy anymore). But as soon as a broken collateral is activated, none of the above actions can be performed.

The activation of a broken collateral is not a problem only for the borrower itself (that cannot repay debt or withdraw collateral anymore) but also a burden for the other suppliers that won't be able to claim their yield given that the borrower won't be able to repay the debt or get liquidated.

Recommendation: Morpho should consider actively checking the liveness of the collateral's oracle when `supplyCollateral` is executed.

Morpho: We had this behavior in mind. we decided against it to keep things simple and:

- A market is as weak as the weakest oracle (and we cannot do anything about it without big changes).
- Oracle reverting is extremely rare (never happened in 2.5y of Morpho Blue). basically that would be a malicious oracle case, but in that case the oracle could as well have returned a very unfair price...

Stermi: Acknowledged.

3.2 Informational

3.2.1 Bulk Informational Finding

Severity: Informational.

Context: (No context files were provided by the reviewer).

(This finding contains elements that refer to `MidnightBundles.sol`, whose additional findings were delivered on a different report).

Description:

1. Consider surrounding with the `""` character in the Midnight's contract natspec everything that is code related (values, variables, function names, contract names and so on). Plenty of times it's challenging to read the natspec, particularly when you use the function's name take which can be confused with the actual verb.
2. `Midnight.sol#L330-L338`: Consider adding additional sanity checks to the `take` function to validate the correctness and coherence of the inputs. For example when `offer.buy=TRUE` we should have `receiverIfMakerIsSeller == address(0)`.
3. `Midnight.sol#L184`: Consider adding a natspec comment to the `consumed` state variable to clarify that such value could be misleading to monitor the funds consumed by a "group" of offers. The user, or an authorized account, can manually increase it.
4. Consider creating periphery contracts to deploy pre-configured Midnight markets. The `setMarketTickSpacing`, `setMarketTradingFee` and `setMarketContinuousFee` functions **require** the market to have already been created.

5. `Midnight.sol#L343`: Consider reverting the `take` action when **both** `offer.maxAssets` and `offer.maxUnits` are equal to zero. The `take` function will **always** revert with the `ConsumedUnits` error unless `units` is equal to zero.
6. `Midnight.sol#L512`: The `onRepay` function does not pass `msg.sender` as part of the input parameter which could be an important piece of information for the `callback` logic.
7. `Midnight.sol#L539`: Consider tracking in the `SupplyCollateral` event if the user has activated a new collateral or emit a custom new event.
8. `Midnight.sol#L566`: Consider tracking in the `WithdrawCollateral` event if the user has deactivated an existing collateral or emit a custom new event.
9. In `Midnight` the `collateral` function returns a `uint128` value given that the collateral amount of a user is stored in `uint128`. The `creditOf` function is instead returning the credit of a user in `uint256` even if the credit is also stored as a `uint128` value. Morpho should consider aligning all these functions to return the same integer type.
10. `EventsLib.sol#L24`: The `callback` input parameter of the `Repay` event should be named `payer` and not `callback`.
11. `MidnightBundles.sol`: Apply the same order of `_forceApproveMax` and `_pullToken` execution across all the bundler's functions.
12. `MidnightBundles.sol`: Consider refactoring the functions in the `MidnightBundles` contract to apply, where possible and needed, a common and coherent code and logic. For example, execute the `_forceApproveMax` and `_pullToken` functions in the same order or define the `loanToken = takes[0].offer.market.loanToken` local variable to be used in the code instead of always accessing it with the long-form reference. There are multiple places where the code can be improved for better maintainability and reading experience.
13. `MidnightBundles.sol#L357`: Align the `_safeApprove` function logic to the one used by similar functions defined in `SafeTransferLib`. Add a custom error when the `require` statement fails.

Recommendation: Morpho should consider implementing all the suggestions listed in the above section.

Stermi:

- 1) Acknowledged by Morpho.
- 2) Fixed by [PR 962](#).
- 3) Fixed by [PR 962](#).
- 4) Acknowledged by Morpho.
- 5) Fixed by [PR 962](#).
- 6) Acknowledged by Morpho.
- 7) Acknowledged by Morpho.
- 8) Acknowledged by Morpho.
- 9) Fixed by [PR 935](#). Now `creditOf`, `debtOf`, `totalUnits`, `withdrawable` and `continuousFee` `Credit` returns `uint128`.
- 10) Fixed by [PR 909](#).
- 11) Fixed by [PR 919](#).
- 12) Fixed by [PR 919](#).
- 13) Fixed by [PR 951](#).

3.2.2 Bulk natspec issues

Severity: Informational.

Context: (No context files were provided by the reviewer).

(This finding contains elements that refer to `MidnightBundles.sol`, whose additional findings were delivered on a different report).

Description:

1. `Midnight.sol#L105-L107`: The natspec of the "AUTHORIZATIONS" sections should be updated to specify that the authorized account/sc can **fully** manage the authorizations for the `onBehalf` user. The authorized account can **add** other accounts but **also remove** existing accounts. The natspec does not mention the removal part of the logic.
2. `Midnight.sol#L160`: consider adding to the list of the assumptions in the "MISC" section that Midnight assumes that both the `buyerCallback` and `sellerCallback` executions will consume as little gas as possible. Theoretically those callback calls could be used to grief `msg.sender` to pay for "extra" gas and make the tx expensive.
3. `Midnight.sol#L164`: `lossFactor` in the MISC natspec comment should be replaced with `lastLossFactor`.
4. `Midnight.sol#L164`: `continuousFeeCredit` and `lastAccrual` should be included in the list of values/functions that could be not up-to-date when fetched.
5. `Midnight.sol#L160`: Consider expanding the "no-op" disclosure in the "MISC" natspec section documenting that operations like `withdrawCollateral` or `repay` can be executed even on non-existing markets (that have no collateral to be withdrawn or debt to be repaid) as long as the `assets / units` (depending on the function signature) is equal to zero. Those "impossible" operations will not revert, create a new market and emit an event.
6. `TickLib.sol#L54`: The natspec of the `TickLib.priceToTick` function contains a typo: "Among the ticks **than** are multiples" → "Among the ticks **that** are multiples".
7. `MidnightBundles.sol`: Consider adding to the general assumption of the `MidnightBundles` contract that the order of the `takes` and the amount `units` configured for each of them matters quite a lot for both the success of the operation and also for how much the operation is going to cost/be profitable (depending on if you are a borrower or a supplier).
8. `MidnightBundles.sol`: Consider adding to the general assumption of the `MidnightBundles` contract that "No-ops are allowed" and "Zero checks are not systematically performed".
9. `MidnightBundles.sol`: The `buyWithUnitsTargetAndWithdrawCollateral` and `buyWithUnitsTargetAndWithdrawCollateral` functions mention the usage of `TakeAmountsLib` saying "@dev Reverts if TakeAmountsLib or ConsumableUnitsLib reverts." but they **do not** use it internally. Remove the mention from the natspec comments.
10. `MidnightBundles.sol`: Consider rewriting the natspec section about "Total receipt". The term and concept are not clear as they are right now.
11. `MidnightBundles.sol#L363`: Consider documenting the special allowance check performed by `MidnightBundles._forceApproveMax`.
12. `HashLib.sol#L51-L52`: The comment "In practice the height is capped at 20 by offerTreeTypeHash" is misleading. The comment is true for the `EcrecoverRatifier` because `HashLib.offerTreeTypeHash(proof.length)` would revert with `length > 20` but it's **not** true for the `SetterRatifier`. The `SetterRatifier` has **NO cap** on the height of the tree.
13. `Midnight.sol#L111-L118`: There are scenarios where the `Minight.take` operation could end up (depending on the `tick`, `price`, `_tradingFee` and rounding) with `buyerAssets` or `sellerAssets` equal to zero. This means that (depending on the `offer` configuration) one of the parts will lose something and the other will gain something. The "ROUNDING" section of the natspec should be updated to cover these scenarios.

14. `Midnight.sol#L27-L28`: The “When some assets become withdrawable before maturity (after a repayment or a liquidation), there is an incentive to take resting sell offers with price < 1 and withdraw instantly” natspec should also include the fee that the `taker` (the one that is buying the units) has to pay to withdraw those units instantly at face value.

Recommendation: Morpho should consider implementing all the suggestions listed in the above section.

Morpho: Acknowledged.

Stermi: Acknowledged.

3.2.3 Trading fees can be set higher as we approach Market's Time To Maturity

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: By looking at the Midnight docs and the Whitepaper we could say that both the Default Trading Fees and the Market Trading Fees assume that the trading fees are an increasing value between breakpoints. But this assumption is not enforced by the `setMarketTradingFee` and `setDefaultTradingFee`. For example the `feeSetter` can freely set the default fee at `index = 0` **higher** than the `index = 6`.

```
uint256 maxTF0 = maxTradingFee(0);
uint256 maxTF0Minus1CBP = maxTradingFee(0) - CBP;
midnight.setDefaultTradingFee(address(loanToken), 0, maxTF0);
midnight.setDefaultTradingFee(address(loanToken), 6, maxTF0Minus1CBP);
```

Without enforcing this requirement a user could end up paying a bigger fee even if the obligation is near or even after maturity.

If not enforcing it (even if by default it is configured to follow the expected behavior) is intended, you should at least make it clear that the `feeSetter` could update the fees (of the market/asset) to change the default expected behavior (lower fee as we approach `ttm = 0`).

Recommendation: Morpho should consider enforcing the expected monotonic fee curves behavior. If non-monotonic fee curves are intended, document explicitly that each breakpoint is independently configurable and that the fee curve is not guaranteed to be monotonic.

Morpho: Acknowledged. At the end we judged not worth fixing. The real guarantee people need is the max fee.

Stermi: Acknowledged.

3.2.4 Consider exposing a `isOfferRatified` function in the Midnight contract

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: Both the `isRatified` functions from the two ratifiers (`EcrecoverRatifier` and `SetterRatifier`) revert when `isRatified` is called by a `msg.sender` that is not Midnight itself. This does not allow an external user to simulate if the execution of `take` would revert because the offer has not been ratified or has been cancelled by the manager of the ratifier.

Recommendation: It could make sense to implement a `isRatified` external function in Midnight that simply calls the ratifier and returns `true` if the offer is valid.

```
function isOfferRatified(Offer memory offer, bytes memory ratifierData) external
↳ returns (bool) {
```

```

    return IRatifier(offer.ratifier).isRatified(offer, ratifierData) ==
    ↪ CALLBACK_SUCCESS;
}

```

By removing this check you are allowing `midnight1.take` to succeed even if the `offer` is using a `ratifier` that does not have `midnight1` as the configured `MIDNIGHT`.

This issue has been further documented in [a new review](#).

3.2.5 Liquidation can seize the whole borrower collateral without paying any debt when collateral price is zero

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: Midnight accepts `oracle.price()` equal to zero as a **valid** price. The main consequence is that a liquidator is allowed to execute a `liquidate` operation to seize the whole borrower's collateral position without paying any.

Recommendation: Morpho should consider reverting the liquidate position when the total amount of debt repaid by the liquidator is equal to zero and the final amount of seized assets is greater than zero.

This scenario should be anyway documented as a special case in the existing natspec documentation of the contract under the "LIVENESS" section.

Morpho: Acknowledged (already flagged by the first Spearbit audit, as "medium"...). Note that there is no real discontinuity with price = 0. price = 1 is probably unfair too, etc etc. basically the spec is "use a good oracle"...

Stermi: Acknowledged.

3.2.6 Liquidation could end up repaying debt without receiving any collateral back

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: The `liquidate` function always rounds against the liquidator.

```

seizedAssets = repaidUnits.mulDivDown(lif, WAD).mulDivDown(ORACLE_PRICE_SCALE,
↪ liquidatedCollatPrice);

```

Because of that it's possible that a liquidator could end up repaying a non-zero debt without receiving any collateral back.

Recommendation: Morpho should consider documenting this scenario under the "LIQUIDATIONS" section of the natspec.

Morpho: Fixed in [PR 936](#).

Stermi: Fix verified.

3.2.7 Consider allowing the owner of the nonce (or an authorized user) to increase the `EcrecoverAuthorizer`'s nonce

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: The `EcrecoverAuthorizer` contract uses the `mapping(address => uint256) public nonce` state variable to validate the authorization requested passed as an input of the `setIsAuthorized` (`Authorization memory authorization, Signature calldata signature`) function.

There could be scenarios where the "owner" of the `nonce` namespace would need to be able to increase the current `nonce[msg.sender]` value to invalidate signatures with that nonce value or "unlock" a "locked" nonce.

The second scenario would happen if an existing authorized user (or the owner of the nonce) de-authorize the `signer` in the scenario where `signer != authorization.authorizer`. When this happens, the `setIsAuthorized` operation for that signature, with that nonce, cannot be executed. All next signature executions, with the incremented nonce, cannot also be executed given the requirement on the nonce to be previously increased.

Recommendation: Morpho should consider introducing a `increaseNonce` function that would allow the owner of the nonce namespace to increase the current nonce value to invalidate existing signatures or unlock future operations.

Morpho: Acknowledged. note that permit, permit2 and Morpho Blue don't have such features. you have to do a signature to increase the nonce.

Stermi: Acknowledged.

3.2.8 Consider tracking the `signer` in the `SetIsAuthorized` event of the `EcrecoverAuthorizer` contract

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: To be able to execute `midnight.setIsAuthorized`, the `EcrecoverAuthorizer.setIsAuthorized` function requires that the "original user" must have called `minight.setIsAuthorized` (`EcrecoverAuthorizer, true, msg.sender`) (or an already authorized user for him).

When `signer != authorization.authorizer` we are in the scenario where `authorization.authorizer` (the "original user") has approved `signer` in the past to behave in his behalf.

But `IMidnight(MIDNIGHT).setIsAuthorized` below is executed by the `EcrecoverAuthorizer` itself and we are **losing** the important information of **who is** in reality approving `authorization.authorized` on behalf of `authorization.authorizer`.

In this case, `msg.sender` is **not** required to be the `signer`. `SetIsAuthorized` should also track the `signer` when we are in the scenario `signer != authorization.authorizer`.

Recommendation: Morpho should consider adding the `signer` address to the `SetIsAuthorized` event emitted by `EcrecoverAuthorizer.setIsAuthorized`.

Morpho: Fixed in PR 966.

Stermi: Acknowledged.

3.2.9 The "ROUNDINGS" should document that different `units` inputs could produce the same results in assets transferred

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: Because of the rounding logic applied by the `take` functions, by providing different values of `units` (diff by 1 wei), the `take` execution could end up (depending on the offer and market state) end up transferring the same amount of assets between the supplier and the borrower.

Because of the rounding logic applied, the user needs to be aware of which is the "best" amount of `units` to buy/sell in a `take` transaction to minimize/maximize (depending on if it's a buy or sell take) the loan token transferred.

Example: we have `buy=TRUE` offer and the seller (the `taker`) executes the `take` operation by specifying 2 different `units` that would result in the **same** amount of assets received by the `taker` (amount borrowed).

This means that 2 different amounts of debt (how much the borrower pays at maturity) generate the same amount of borrowed amount. In one case the borrower has a loss of `1 wei` of loan token value. By specifying the "correct" `units` that loss could have been avoided.

Proof of Concept:

- In scenario 1 the lender purchases `105.168727447881007895` units.
- In scenario 2 the lender purchases `105.168727447881007894` units (1 wei less).

But in **BOTH** scenarios the lender loans 100 assets to the borrower this mean that, because of rounding, in the second scenario the lender will lose 1 wei of face value (final credit to be redeemed) taking 1 wei of loss.

```
// SPDX-License-Identifier: GPL-2.0-or-later
// Copyright (c) 2025 Morpho Association
pragma solidity ^0.8.0;

import {IMidnight, Market, Offer, CollateralParams} from
↳ "../src/interfaces/IMidnight.sol";
import {
    IBuyCallback,
    ISellCallback,
    ILiquidateCallback,
    IRepayCallback,
    IFlashLoanCallback
} from "../src/interfaces/ICallbacks.sol";
import {Midnight} from "../src/Midnight.sol";
import {IdLib} from "../src/libraries/IdLib.sol";

import {ERC20} from "../erc20s/ERC20.sol";
import {Oracle} from "../helpers/Oracle.sol";
import {RevertingOracle} from "../helpers/RevertingOracle.sol";
import {BaseTest, MAX_TEST_AMOUNT} from "../BaseTest.sol";
import {
    MAX_COLLATERALS,
    MAX_COLLATERALS_PER_BORROWER,
    MAX_CONTINUOUS_FEE,
    WAD,
    CBP,
    ORACLE_PRICE_SCALE,
    TIME_TO_MAX_LIF,
    CALLBACK_SUCCESS
} from "../src/libraries/ConstantsLib.sol";
import "../src/libraries/ConstantsLib.sol";
import {UtilsLib} from "../src/libraries/UtilsLib.sol";
import {ConsumableUnitsLib} from "src/periphery/ConsumableUnitsLib.sol";
import "forge-std/console.sol";
import {TickLib, MAX_TICK} from "../src/libraries/TickLib.sol";

contract _STakeRounding is BaseTest {
    using UtilsLib for uint256;
```

```

Market internal market;
bytes32 internal id;

function setUp() public override {
    super.setUp();

    market.loanToken = address(loanToken);
    market.maturity = block.timestamp + 100;
    market.collateralParams
        .push(
            CollateralParams({
                token: address(collateralToken1),
                lltv: 0.77e18,
                maxLif: maxLif(0.77e18, 0.25e18),
                oracle: address(oracle1)
            })
        );
    market.collateralParams
        .push(
            CollateralParams({
                token: address(collateralToken2),
                lltv: 0.77e18,
                maxLif: maxLif(0.77e18, 0.25e18),
                oracle: address(oracle2)
            })
        );
    market.collateralParams = sortCollateralParams(market.collateralParams);
    market.rcfThreshold = 0;

    vm.prank(borrower);

    midnight.setIsAuthorized(address(this), true, borrower);

    id = midnight.touchMarket(market);
}

function testBuyRonding() public {
    uint256 maxAssets = 100e18;

    deal(address(loanToken), lender, 100_000_00e18);

    (uint256 tick, uint256 actualPrice) = calcTickAndPrice(id, 0.95e18);

    Offer memory offer;
    offer.buy = false;
    offer.maker = borrower;
    offer.ratifier = address(ecrecoverRatifier);
    offer.maxAssets = maxAssets;
    offer.market = market;
    offer.expiry = block.timestamp + 200;
    offer.tick = tick;
    offer.group = keccak256(abi.encode("borrower-buy-1000-usdc"));

    uint256 consumableUnits =
        ↪ ConsumableUnitsLib.consumableUnits(address(midnight), id, offer);

    uint256 snapshot = vm.snapshotState();
    (uint256 buyerAssets1, uint256 sellerAssets1) = _consume(offer,
        ↪ consumableUnits);
    uint256 effectiveConsumed1 = midnight.consumed(offer.maker, offer.group);
    uint256 lenderCredit1 = midnight.creditOf(id, lender);
    uint256 borrowerDebt1 = midnight.debtOf(id, borrower);
    vm.revertToState(snapshot);
}

```

```

(uint256 buyerAssets2, uint256 sellerAssets2) = _consume(offer,
    ↪ consumableUnits-1);
uint256 effectiveConsumed2 = midnight.consumed(offer.maker, offer.group);
uint256 lenderCredit2 = midnight.creditOf(id, lender);
uint256 borrowerDebt2 = midnight.debtOf(id, borrower);

console.log('consumableUnits %18e', consumableUnits);
console.log('buyerAssets1 %18e', buyerAssets1);
console.log('buyerAssets2 %18e', buyerAssets2);
console.log('sellerAssets1 %18e', sellerAssets1);
console.log('sellerAssets2 %18e', sellerAssets2);
console.log('effectiveConsumed1 %18e', effectiveConsumed1);
console.log('effectiveConsumed2 %18e', effectiveConsumed2);
console.log('lenderCredit1 %18e', lenderCredit1);
console.log('lenderCredit2 %18e', lenderCredit2);
console.log('borrowerDebt1 %18e', borrowerDebt1);
console.log('borrowerDebt2 %18e', borrowerDebt2);

// In scenario 1 the lender purchase `105.168727447881007895` units
// In scenario 2 the lender purchase `105.168727447881007894` units (1 wei
↪ less)
// but in BOTH scenarios the lender loan 100 assets to the borrower
// this mean that, because of rounding, in the second scenario the lender will
↪ lose 1 wei of
// face value (final credit to be redeemed) taking 1 wei of loss.

assertEq(buyerAssets1, buyerAssets2);
assertEq(sellerAssets1, sellerAssets2);
assertEq(lenderCredit1, lenderCredit2 + 1);
assertEq(borrowerDebt1, borrowerDebt2 + 1);
}

function _consume(Offer memory offer, uint256 units) public returns (uint256
    ↪ buyerAssets, uint256 sellerAssets) {
    if( offer.buy ) {
        collateralize(market, borrower, units);
        (buyerAssets, sellerAssets) = take(units, borrower, offer);
    } else {
        collateralize(market, borrower, units);
        (buyerAssets, sellerAssets) = take(units, lender, offer);
    }
}
}

```

Recommendation: Morpho should improve the documentation of the "ROUNDINGS" section of the natspec by covering this scenario.

Morpho: Fixed in PR 936.

Stermi: Fix verified, it would have been preferable having a more explicit natspec (relative to the consequences) than the current implementation.

3.2.10 Consider passing the "opposite" side data information to the `onBuy` and `onSell` callbacks

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: The gates (when enabled) validates the `buyer` / `seller` . Gates are defined at the `market` level and not at the offer level. The `maker` of the offer could (and probably it's the vast majority of the

cases) be different from the "owner" of the market. And you cannot even require or force the `maker` to create a new market just to be able to validate the buyer/seller because this would mean to further fragmenting the Midnight liquidity which is not ideal.

The ratifiers only validate that the `offer` is a "valid" offer that has been signed/authorized by the `maker` (or by someone who has been authorized by the `maker`) but **cannot** perform any validation on the `take` instance (offer + input params) or the "state" of the market (trading fees/continuous fees, loss factor).

The same can be said for the `onBuy` or `onSell` callbacks. When `take` is performed with `offer`, `buy=true` the `maker` is the `buyer` but the `onBuy` callback (the one that can be configured by the maker) does not pass as inputs the "opposite side of the operation's data", not even who is the `seller` (the taker). When `take` is performed with `offer.buy=false` the `maker` is the `seller` but the `onSell` callback (the one that can be configured by the maker) does not pass as inputs the "opposite side of the operation's data", not even who is the `buyer` (the taker).

Let's say that as a `maker` (which is "passive", is not the one executing the `take` operation) a user could have to allow only specific accounts to take the offer (because of security compliance or legal reasons).

How can the `maker` abide those obligations? The same could be said also for the `taker` when the operation is executed by an authorized user but this scenario is covered by the natspec.

Recommendation: Morpho should consider enhancing the `onBuy` and `onSell` inputs by including the "opposite" side data information of the `take` operation.

Morpho: Acknowledged. This is an intentional design choice to prevent people from having callbacks whose execution depends on the taker, as this would complexify routing significantly. Also we don't see interesting use-cases with that.

Stermi: Acknowledged.

3.2.11 Considerations on the lenders hostile and adversarial environment

Severity: Informational.

Context: (No context files were provided by the reviewer).

Description: As soon as a borrower repays his debt partially/fully via `repay` or is liquidated partially/fully via `liquidate`, the operation will increase "globally" (at market level) the amount of units that can be withdrawn for a **specific** market.

What are the reasons why a supplier wants to **stay** in the market? Unlike other lending products, Midnight provides **no incentive** to stay as a lender if some debt has been repaid/liquidated. We could even say that the `pendingFee` is a **disincentive** for the supplier to stay in the market.

This means that as soon as some debt is cleaned via `repay` / `liquidate`, **every** supplier is incentivized to withdraw as much `loanToken` as possible by burning their `credit` and pay as little `pendingFee` as possible.

This creates a hostile and adversarial environment for lenders because the available `_marketState.withdrawable` to be withdrawn is **globally shared** at "market level". Every supplier will try to withdraw as soon as possible frontrunning each other and paying higher gas fees to be the "first one".

In addition to that the Natspec correctly mentions another scenario that aggravates the existing situation: the platform also enables arbitrage opportunities where non-existing lenders will, as soon as they see `repay/liquidate` events, purchase credits to be instantly withdrawn to get a profit (depending on market value for those credits + the transaction fee).

Recommendation: Morpho should consider improving the existing natspec documentation to underline the hostile and adversarial environment that the suppliers could end up in.

Morpho: Acknowledged. We are ok with this because if you don't participate to the race you still get an ok value from the protocol (what you would have if we completely prevented this race). Basically it doesn't

harm you. The only real harm that is done is that the early withdrawer pay a bit less continuousFee... but we are ok with this too.

***Stermi:** Acknowledged.